HW 2 16 points

Name (in Korean if possible): 이경민
Student ID (8-digits): 22100487

1. (2 points) The `Merge()` function that takes two sorted subarrays of `A` and merges them into a single sorted subarray of A can be implemented as below. What is the running time of the `Merge()` function in terms of Θ-notation? Show your complete problem-solving process.

```
// pseudocode
MergeSort(A, left, right) {
  if (left < right) {
    mid = floor((left + right) / 2);
    MergeSort(A, left, mid);
    MergeSort(A, mid+1, right);
    Merge(A, left, mid, right);
  }
}
```

```
// pseudocode
Merge(A, left, mid, right) {
  i = left
  j = mid + 1
  m = 0
  create array M[0 ... right-left]
  while (i ≤ mid and j ≤ right) {
    if (A[i] ≤ A[j]) {
      M[m] = A[i]
      i = i + 1
    }
    else {
      M[m] = A[j]
      j = j + 1
    }
    m = m + 1
  }
  while (i ≤ mid) {
    M[m] = A[i]
    i = i + 1
    m = m + 1
  }
  while (j ≤ right) {
    M[m] = A[j]
    j = j + 1
    m = m + 1
  }
  for k = 0 to right-left {
    A[left + k] = M[k]
  }
}
```

$\Theta(n)$

```
Merge(A, left, mid, right) {
  i = left
  j = mid + 1
  m = 0
  create array M[0 ... right-left]
  while (i ≤ mid and j ≤ right) {
    if (A[i] ≤ A[j]) {
      M[m] = A[i]
      i = i + 1
    }
    else {
      M[m] = A[j]
      j = j + 1
    }
    m = m + 1
  }
  while (i ≤ mid) {
    M[m] = A[i]
    i = i + 1
    m = m + 1
  }
  while (j ≤ right) {
    M[m] = A[j]
    j = j + 1
    m = m + 1
  }
  for k = 0 to right-left {
    A[left + k] = M[k]
  }
}
```

Variable initialization : $\Theta(1)$

$n \geq k$

One element is moved to array M each time loops run.

If, first while loop, k elements are moved to the array M

(n-k) elements are moved in second and third while loops

$k + (n-k) = n \rightarrow \Theta(n)$

$\rightarrow n-1$

$k = 0 \sim (n-1) \rightarrow n$ times.

$\therefore \Theta(n)$

$\Theta(1) + \Theta(n) + \Theta(n) = \Theta(n)$

Transparency

i. I checked with the AI how to calculate the running time.

ii. https://chatgpt.com/share/69c11d5b-b718-8012-aadb-e8db7e6837b7

2. (Bonus 1 point) Prove the correctness of `MergeSort()` by mathematical induction. Show your complete problem-solving process.

Base case : $n=1$ → left=right. → function does nothing.
↳ It's already sorted. $P(1)$ is true.

Inductive hypothesis:
For n greater than the base case,
assume that the formula holds true for all $1 \le k < n$

$\boxed{\begin{array}{l} 1 \le k_1 < n \\ 1 \le k_2 < n \end{array}}$

Proof of goal statement:

1. Since $n > 1$, left < right : true → excute if statement.
2. calculate mid. → Devide array A which size is n into two smaller subarrays.
   A[left... mid] → size k_1
   A[mid+1 ... right] → size k_2
3. Since $k_1 < n$: Mergesort (A, left, mid) will sort the left subarray.
4. Since $k_2 < n$: Mergesort (A, mid+1, right) will sort the right subarray.
5. Excute Merge (A, left, mid, right), → combine two sorted subarrays correctly.

So, the function Mergsort sorts the array of size n.

Conclusion. $P(n)$ is true.

Transparency

i. I gave the results to GPT and asked if my proof had gone in the right direction.

ii. https://chatgpt.com/share/69c127d8-a164-8012-ba12-95847301cded

3. (4 points) Prove that $T(n) = O(n^2)$ where $T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$ by using the substitution method. Show your complete problem-solving process.

$$T(n) = \begin{cases} \Theta(1) & \text{if } n=1 \\ 3T(n/4)+cn^2 & \text{if } n>1 \end{cases}$$

GUESS $T(n) = O(n^2)$

$\hookrightarrow T(\frac{n}{4}) = O(\frac{n^2}{16})$

Let's prove that $T(n) \le dn^2$ for some $d>0$.

$$T(n) = 3T(\tfrac{n}{4}) + cn^2$$
$$= 3\left(d \cdot \frac{n^2}{16}\right) + cn^2$$
$$= \left(\frac{3}{16}d + c\right)n^2$$
$$\le dn^2 \quad \text{for} \quad d \ge \frac{16}{13}c$$

$\frac{3}{16}d + c \le d$

$c \le \frac{13}{16}d$

$d \ge \frac{16}{13}c$

Transparency

i. I have slight concern about unfamiliar d value. So I checked it using AI.

ii. https://chatgpt.com/share/69c12a09-21bc-8012-9dce-a9af077578ee

4. (2 points) What is the tight bound for $T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$ by using Master Method? Show your complete problem-solving process.

$\Theta(n^2)$

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 3T(n/4) + cn^2 & \text{if } n > 1 \end{cases}$$

$a=3$
$b=4$

$0 < \log_4 3 < 1 \quad (\log_4 4 = 1)$

$n^{\log_4 3}, \quad f(n) = cn^2 \quad (\log_4 3 < 2)$

$n^2 = \Omega(n^{\log_4 3 + \epsilon}) \quad \epsilon > 0$

$3f(\frac{n}{4}) \le d\, f(n)$ for $d < 1$

$3 \cdot c \cdot (\frac{n}{4})^2 \le d \cdot cn^2$

$\frac{3}{16} cn^2 \le d \cdot cn^2$

$\frac{3}{16} \le d$ true

$\therefore T(n) = \Theta(f(n)) = \Theta(cn^2)$

$= \Theta(n^2)$

Transparency

i. I studied the Master Method results and the situations in which three cases are used.

ii. https://chatgpt.com/share/69bee7c0-e378-8012-bdbb-aa55d85a71c8

5. (2 point) (continued from the previous problem) $T(n) = \omega(n^2)$? Show your complete problem-solving process.

$$T(n) \neq \omega(n^2)$$

little omega (ω) represents a strictly lower bound.

$T(n) = \omega(n^2)$ means $T(n)$ grows strictly faster than n^2 $(n \to \infty) \Rightarrow \lim_{n\to\infty} \frac{T(n)}{n^2} = \infty$

P4. We already get $T(n) = \Theta(n^2)$

$\hookrightarrow \lim_{n\to\infty} \frac{T(n)}{n^2} = C$

So, $T(n) \neq \omega(n)$

Transparency

i. I checked the definition of small omega once again using AI.

ii. https://chatgpt.com/share/69c0f704-8400-8012-a254-8d62d7791d73

6. Follow the instructions below.

A. Implement InsertionSort() function. (Use the given template code.)

Transparency
i. This problem was solved by referring only to the pseudocode in lecture named “01 Getting Started.pdf” without using AI.

B. Implement Merge() function. (Use the given template code.)

Transparency
i. This problem was solved by referring only to the pseudocode in problem 1 of HW2 without using AI.

C. Print the execution time (in usec) of each sorting function and print them.

i. The result must look like this, but numbers would be different.

```
The result is sorted in ascending order.
It took 30 usec by Insertion Sort
The result is sorted in ascending order.
It took 49 usec by Merge Sort
```

```
gyeongmin@igyeongmin-ui-MacBookAir  ~/HGU/26-1/algorithms/HW/HW2  main ±  g++ HW_6.cpp
gyeongmin@igyeongmin-ui-MacBookAir  ~/HGU/26-1/algorithms/HW/HW2  main ±  ./a.out
The result is sorted in ascending order.
It took 26 usec by Insertion Sort
The result is sorted in ascending order.
It took 58 usec by Merge Sort
gyeongmin@igyeongmin-ui-MacBookAir  ~/HGU/26-1/algorithms/HW/HW2  main ±
```

D. Calculate and print the ratio of the execution times of the two sorting functions (Ex: MergeSort execution time / InsertionSort execution time) for the input size $n = 10, 50, 100, 500, 1000, 2000, \text{and } 3000$.

i. Please note that the ratio may vary from run to run even with the same n depending on the server computer's condition.

ii. Try running your code at least three times and calculate the average value for each n case.

E. (4 points) Fill the following table with the result above.

n	10	50	100	500	1000	2000	3000
Ratio (average)	6.33	3.77	2.42	0.61	0.29	0.14	0.09

	(Insertion, Merge)				
n=10	(1, 7)	(1, 6)	(1, 6)	→	6.3333...
	7	6	6		6.33
n=50	(10, 33)	(7, 28)	(7, 28)	→	3.7616...
	3.3	4	4		3.77
n=100	(25, 58)	(22, 53)	(21, 53)	→	2.41666...
	2.32	2.41	2.52		2.42
n=500	(591, 382)	(502, 289)	(516, 314)	→	0.61333...
	0.65	0.58	0.61		0.61
n=1000	(2469, 721)	(1949, 521)	(2094, 606)	→	0.29333...
	0.29	0.30	0.29		0.29
n=2000	(9451, 1262)	(8654, 1289)	(7643, 995)	→	0.13 6166...
	0.13	0.15	0.13		0.14
n=3000	(18993, 1633)	(15617, 1426)	(16429, 1479)	→	0.09
	0.09	0.09	0.09		0.09

F. (Bonus 1 point) Find n (approximately) where the insertion sort gets slower than the merge sort i.e. where the ratio becomes around 1.

About n = 300

G. (2 points) Explain why insertion sort takes longer time than merge sort does when the input size is large. Use the growth-of-function notations (Θ or O) for your explanation.

Insertion sort : runs in $O(n^2)$

Merge sort : runs in $O(n \log n)$

If n becomes large, n^2 grows much faster than $n \log n$.

$n \log n$ is asymtotically smaller than n^2.

So, Insertion sort takes longer time than merge sort.

Transparency

i. I confirmed the concept I know.

ii. https://chatgpt.com/share/69c15770-4804-8012-bdb9-33f314aaee1f

```
<Template Code>
#include <iostream>
#include <vector>
#include <cstdlib>
```

```
#include <ctime>
#include <cstring>
#include <chrono>

using namespace std;

void InsertionSort(vector<int>& A);
void MergeSort(vector<int>& A, int left, int right);
void Merge(vector<int>& A, int left, int mid, int right);

void GenerateRandomIntegers(vector<int>& A);
bool IsSorted(const vector<int>& A, bool ascendingOrder);
void CheckTheResult(const vector<int>& A, bool ascendingOrder);
void PrintArray(const vector<int>& A);
long long GetCurrentUsec();

int main() {
    int n = 100; // change this number
    vector<int> A(n);
    GenerateRandomIntegers(A);

    vector<int> B = A;

    // print input
    // cout << "Input : ";
    // PrintArray(A);
    // CheckTheResult(A, true);

    long long t1, t2;

    // Insertion Sort
    t1 = GetCurrentUsec();
    InsertionSort(A);
    t2 = GetCurrentUsec();
    // cout << "Output : ";
    // PrintArray(A);
    CheckTheResult(A, true);
    cout << "It took " << (t2 - t1) << " usec by Insertion Sort" << endl;

    // Merge Sort
    t1 = GetCurrentUsec();
    MergeSort(B, 0, n - 1);
    t2 = GetCurrentUsec();
    // cout << "Output : ";
    // PrintArray(B);
    CheckTheResult(B, true);
    cout << "It took " << (t2 - t1) << " usec by Merge Sort" << endl;
```

```
    return 0;
}

void GenerateRandomIntegers(vector<int>& A) {
    srand(static_cast<unsigned int>(time(nullptr)));
    for (int& x : A) {
        x = rand() % 100;
    }
}

void InsertionSort(vector<int>& A) {
    int n = A.size();
    for(int i = 1; i < n; i++) { // A[0] is already sorted. From 1 to n - 1;
        int key = A[i];
        int j = i - 1;

        while (j > 0 && A[j] > key) {
            A[j + 1] = A[j];
            j--;
        }

        A[j + 1] = key;
    }
}

void MergeSort(vector<int>& A, int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        MergeSort(A, left, mid);
        MergeSort(A, mid + 1, right);
        Merge(A, left, mid, right);
    }
}

void Merge(vector<int>& A, int left, int mid, int right) {
    int i = left;
    int j = mid + 1;
    int m = 0;
    int n = right - left + 1;
    vector<int> M(n);

    while(i <= mid && j <= right) {
        if(A[i] <= A[j]){
            M[m] = A[i];
            i++;
        } else {
            M[m] = A[j];
            j++;
```

```
        }
        m++;
    }
    while(i <= mid){
        M[m] = A[i];
        i++;
        m++;
    }
    while(j <= right){
        M[m] = A[j];
        j++;
        m++;
    }
    for(int k = 0; k < n; k++) {
        A[left + k] = M[k];
    }
}

void PrintArray(const vector<int>& A) {
    cout << "{";
    for (int x : A) {
        cout << x << ", ";
    }
    cout << "}" << endl;
}

long long GetCurrentUsec() {
    auto now = chrono::high_resolution_clock::now();
    auto us =
chrono::duration_cast<chrono::microseconds>(now.time_since_epoch());
    return us.count();
}

void CheckTheResult(const vector<int>& A, bool ascendingOrder) {
    cout << "The result is ";
    if (!IsSorted(A, ascendingOrder)) {
        cout << "not ";
    }
    cout << "sorted in ";
    if (ascendingOrder) {
        cout << "ascending";
    } else {
        cout << "descending";
    }
    cout << " order." << endl;
}

bool IsSorted(const vector<int>& A, bool ascendingOrder) {
```

```
    int n = static_cast<int>(A.size());
    for (int i = 0; i < n - 1; i++) {
        if (ascendingOrder) {
            if (A[i] > A[i + 1]) return false;
        } else {
            if (A[i] < A[i + 1]) return false;
        }
    }
    return true;
}
```

For Genius Track students

Write your reasons for challenging the bonus problem(s) and your thoughts here.

Transparency Track

The first step is to implement InsertionSort() and MergeSort(). When I just thought about it, I assumed I could do it easily, but now that I have actually attempted it, I am certain it would have been difficult without pseudocode.I also feel the need to thoroughly study the concepts behind methods for calculating running time, such as the substitution method and the Master method. Of course, the same applies to proofs using mathematical induction.

Appendix

How to compile and run your c++ code:

1. You may use any tool if you already have.
2. Or go to Google Colab and write down the code in this way.
   A. Write your code like below in the first cell but you must include `%%writefile program.cpp` at the top of the code. Then, run the cell.

```
%%writefile program.cpp
#include <iostream>
#include <sys/time.h>

int main() {
    // get the current time (moment 1)
    timeval tv;
    gettimeofday(&tv, nullptr);
    long start = tv.tv_sec * 1000000L + tv.tv_usec;

    // a task that takes time
    int cnt = 0;
    for (int i = 0; i < 1000; i++) {
        cnt++;
    }
    std::cout << "cnt: " << cnt << std::endl;

    // get the current time (moment 2)
    gettimeofday(&tv, nullptr);
    long end = tv.tv_sec * 1000000L + tv.tv_usec;

    // calculate the time difference between the two moments in
microseconds
    long difftime = (end - start);

    // print the result
    std::cout << "the time difference: " << difftime << " usec"
<< std::endl;

    return 0;
}
```

   B. Write the following code in the second cell. Then, run the cell.

```
%%shell
g++ program.cpp -o execute
./execute
```